

PROJE 16 / 20 · ÇALIŞAN TEMEL SÜRÜM

API Gateway ve Token Bucket

API anahtarına göre kota uygula; sabit yerel üst servise gerçek HTTP proxy isteği gönder.

Yerel upstream · token bucket

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.

DYFULL-STACK LAB / 16 / 20Deniz DemoÇıkış

PORTFOLYO PROJESİ · ÇALIŞAN TEMEL SÜRÜM

API Gateway ve Token Bucket

API anahtarına göre kota uygula; sabit yerel üst servise gerçek HTTP proxy isteği gönder.

Yerel upstream · token bucket

API anahtarı oluştur

İstemci adı

Token kapasitesi

Oluştur

Her saniye 1 token eklenir; kapasite aşılmaz. Kota her anahtar için ayrıdır.

Proxy test konsolu

X-API-Key

GET /gateway/demo

Hızlı tıklamalarla HTTP 429 yanıtını gözlemler.

Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

API anahtarı üretme/iptal · Hash olarak anahtar saklama · Kapasite ve saniyede 1 token · HTTP 429 ve Retry-After · İstek günlüğü

Bu sürümde olmayanlar

Tek sabit yerel upstream vardır; kullanıcı URL giremez. Gateway yalnız GET destekler. Dağıtık Redis kotası, IP engelleme ve çoklu route yönetimi yok.

DENİZ YARDIMCI · FULL-STACK LAB · 23.09.2026

1

02 / Backend ve veri modeli

Her istekte $\text{tokens} = \min(\text{capacity}, \text{oldTokens} + \text{elapsedSeconds})$ hesaplanır; token varsa bir azaltılır. Okuma/yazma SQLite transaction içindedir. Kota dolunca 429 döner, üst servise gidilmez. Anahtar gizli değeri yalnız oluştururken gösterilir; veritabanında hash tutulur. İptal edilmiş anahtar çalışmaz.

Gerçek SQL şeması

```
CREATE TABLE api_keys(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),name
TEXT,secret_hash TEXT UNIQUE,capacity INTEGER,tokens REAL,updated INTEGER,active INTEGER
DEFAULT 1);
```

```
CREATE TABLE requests(id INTEGER PRIMARY KEY,key_id TEXT REFERENCES api_keys(id),status
INTEGER,at TEXT);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekiliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametrelili SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/keys	Gövde yok	200
POST /api/keys	JSON: capacity, name	201
DELETE /api/keys/:id	Gövde yok	200
GET /api/requests	Gövde yok	200
GET /gateway/demo	Gövde yok	200

Somut istek örneği

```
POST /api/keys
{"name":"Client A","capacity":3}
201 {"id":"...", "secret":"<one-time-key>"}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- Kapasite 1 için hızlı iki isteğin ikincisi 429 almalı.
- 429 yanıtı Retry-After: 1 içermeli.
- Liste endpoint'i anahtar hash değerini bile döndürmemeli.
- İptal edilen anahtar proxy isteği yapamamalı.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const key=await post('/keys',{name:'Client',capacity:1},201);
const one=await fetch(base+'/gateway/demo',{headers:{'x-api-key':key.secret}});assert.equal(one.status,200);await one.json();
const two=await fetch(base+'/gateway/demo',{headers:{'x-api-key':key.secret}});assert.equal(two.status,429);assert.equal(two.headers.get('retry-after'),'1');await two.json();
assert.ok(!(await get('/keys'))[0].secret_hash);
await call('/api/keys/'+key.id,'DELETE',undefined,200);const revoked=await fetch(base+'/gateway/demo',{headers:{'x-api-key':key.secret}});assert.equal(revoked.status,404);await revoked.json();
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

Kapasiteyi 1 yap, arka arkaya iki test isteği gönder. İlk 200, ikinci 429 olmalı. Bir saniye sonra tekrar dene.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Birden çok güvenilir upstream ekle
- 2. Yöntem ve path allowlist tanımla
- 3. Redis atomik script ile dağıtık kota kur
- 4. Gecikme ve hata grafikleri ekle

Anlamalı commit sırası

```
git add .
git commit -m "chore: add api-gateway baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Token bucket ile fixed window farkı nedir? Aynı anahtar iki sunucuda kullanıldığında kota nasıl ortak tutulur?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.